第三部分

基本算法设计方法

主讲人：孙歧峰

联系电话：13864717907

Q Q号码：274342509

邮　　箱：sunqf@upc.edu.cn

提纲

CONTENTS

3.1 穷举法

3.2 归纳法

3.3 迭代法

3.4 递归法

3.5 递推式计算

3.1 穷举法

3.1.1 穷举法概述

1. 什么是穷举法

- 穷举法又称枚举法或者列举法，是一种简单而直接地解决问题的方法。
- 基本思想是先确定有哪些穷举对象和穷举对象的顺序，按穷举对象的顺序逐一列举每个穷举对象的所有情况，再根据问题提出的约束条件检验哪些是问题的解，哪些应予排除。

常用的列举方法

- 顺序列举。是指答案范围内的各种情况很容易与自然数对应甚至就是自然数，可以按自然数的变化顺序去列举。
- 排列列举。有时答案的数据形式是一组数的排列，列举出所有答案所在范围内的排列，为排列列举。
- 组合列举。当答案的数据形式为一些元素的组合时，往往需要用组合列举。组合是无序的。

穷举法的作用

- 理论上讲穷举法可以解决可计算领域中的各种问题。
- 在实际应用中，通常要解决的问题规模不大，用穷举法设计的算法其运算速度是可以接受的。
- 举法算法一般逻辑清晰，编写的程序简洁明了。
- 穷举法算法一般不需要特别证明算法的正确性。
- 穷举法可作为某类问题时间性能的底限，用来衡量同样问题的更高效率的算法。

2. 穷举法框架

```
void Exhaustive(x,n,y,m)
{   for (int i=1;i<=n;i++)                    //枚举x的所有可能的值
      for (int j=1;j<=m;j++)                  //枚举y的所有可能的值
      {  …
         if (p(x[i],y[j]))
            输出一个解;
         …
      }
}
```

- x和y所有可能的搜索范围是笛卡尔积即（$[x_1, y_1]$，$[x_1, y_2]$，…，$[x_1, y_m]$，…，$[x_n, y_1]$，$[x_n, y_2]$，…，$[x_n, y_m]$）。
- 这样的搜索范围可以用一棵树表示，称为解空间树。

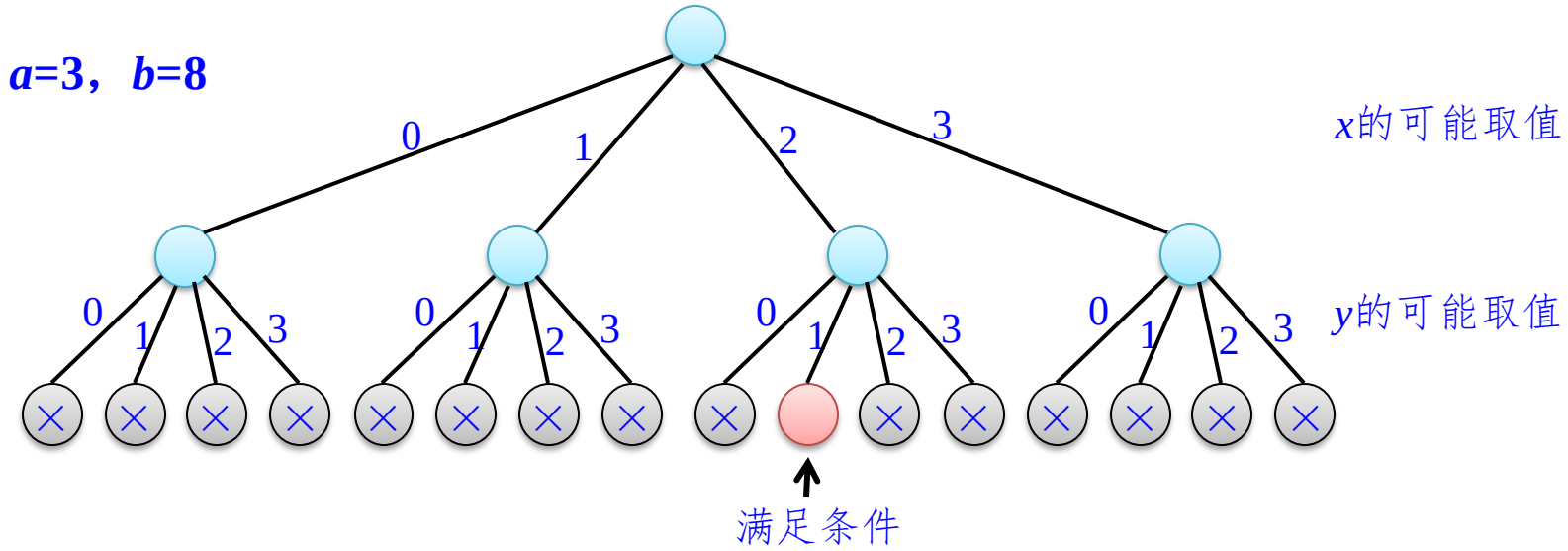
【例3-1】鸡兔同笼问题。现有一笼子，里面有鸡和兔子若干只，数一数，共有a个头，b条腿。设计一个算法求鸡和兔子各有多少只？

解

- x表示鸡的只数，y表示兔的只数，那么穷举对象就是x和y，假设穷举对象的顺序是先x后y（本问题中也可以先y后x）。
- x和y的取值范围都是$0\sim a$，约束条件

  $p(x，y)$为x+y=a && 2x+4y=b。

```
void solve1(int a,int b)
{    for(int x=0;x<=a;x++)
          for(int y=0;y<=a;y++)
          {    if(x+y==a && 2*x+4*y==b)
                    printf("x=%d,y=%d\n",x,y);
          }
}
```

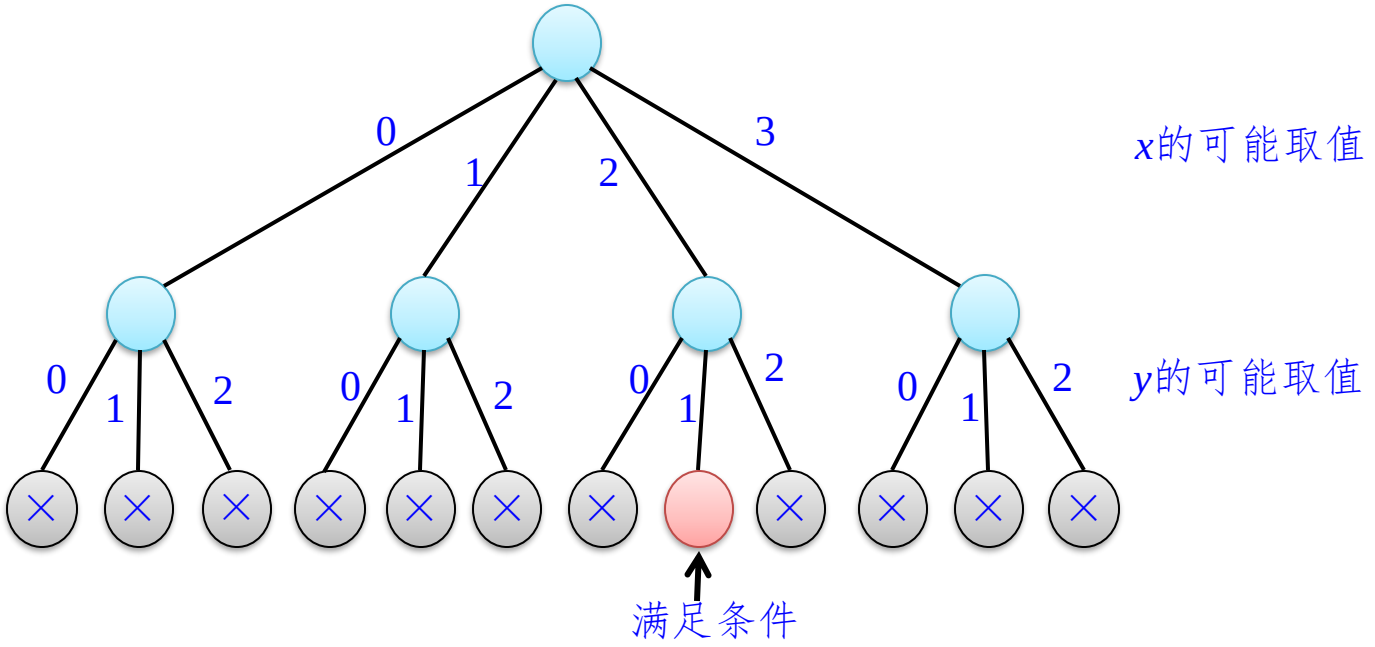


解空间树中共21个结点，显然结点个数越多时间性能越差！

优化：鸡的只数最多为min(a，b/2)，兔的只数最多为min(a，b/4)

```
void solve2(int a,int b)
{    for(int x=0;x<=min(a,b/2);x++)
          for(int y=0;y<=min(a,b/4);y++)
          {    if(x+y==a && 2*x+4*y==b)
                    printf("x=%d,y=%d\n",x,y);
          }
}
```

以a=3，b=8为例，x的取值范围是0～3，y的取值范围是0～2



共17个结点！



尽管穷举法算法通常性能较差，但可以以它为基础进行优化继而得到高性能的算法，优化的关键是能够找出求解问题的优化点！

【例3-2】素数个数问题。给定两个均含n个正整数的数组a和b，其中整数元素的范围是2到20000，求出每对$a[i]$和$b[i]$（$0 \leqslant i < n$）之间的素数个数（含$a[i]$和$b[i]$）。

解

解法1：采用穷举法。设计isPrime(x)算法判断x是否是素数，Count1(a，b)求整数a到b之间的素数个数。在此基础上设计求解算法solve(a，b，n)求每对$a[i]$和$b[i]$（$0 \leqslant i < n$）之间的素数个数。

素数的判断方法：

在一般领域，对正整数n，如果用2到$\sqrt{n}$之间的所有整数去除，均无法整除，则n为质数。

试除法O(sqrt(n))

```
bool isPrime(int x)                    //判断x是否是素数
{    for (int i=2;i<=(int)sqrt(x);i++)
         if (x%i==0)                   //x能够被i整除
              return false;
     return true;
}

int Count1(int a,int b)                //求a到b的素数个数
{    if (a>b) return 0;
     int cnt=0;
     for(int x=a;x<=b;x++)
         if(isPrime(x))
              cnt++;
     return cnt;
}

void solve1(int a[],int b[],int n)     //求解算法
{    for(int i=0;i<n;i++)
         printf("  %d-%d之间的素数个数=%d\n",a[i],b[i],Count1(a[i],b[i]));
}
```

solve1算法的最坏时间复杂度为 $O(mn\sqrt{m})$ 。

解法2：优化。设计一个整数数组prime，其中prime[*i*]表示整数*i*是否是素数，初始时置prime所有元素为1，采用素数筛选法（若*i*是素数，则*i*的倍数一定不是素数）求出所有的非素数。

再将prime转换为前缀和，即将prime[*i*]由原来表示整数*i*是否是素数转换为表示小于等于*i*的素数个数，转换公式如下：

prime[i]+=prime[*i*-1]

这样*a*到*b*之间的素数个数为prime[*b*]-prime[*a*-1]。

```
int prime[MAXD];                          //prime[i]=1表示i是素数
void Init()                               //求出prime数组
{   for(int i=0;i<=MAXD;i++)
        prime[i]=1;
    prime[0]=prime[1]=0;
    for(int i=2;i<=MAXD;i++)
    {   if(prime[i])                      //若i是素数
        {   for(int j=2*i;j<=MAXD;j+=i)   //则i的倍数都不是素数
                prime[j]=0;
        }
    }
}

void solve2(int a[],int b[],int n)        //求解算法
{   Init();
    for(int i=2;i<=MAXD;i++)              //prime[i]累计小于等于i的素数个数
        prime[i]+=prime[i-1];
    for(int i=0;i<n;i++)
        printf("  %d-%d之间的素数个数=%d\n",a[i],b[i],prime[b[i]]-prime[a[i]-1]);
}
```

这里用的埃氏筛而非欧拉筛，所以应该是 n + mloglogm

solve2算法的最坏时间复杂度为O(*m+n*)

3.2 归纳法

3.2.1 归纳法概述

1. 什么是数学归纳法

第一数学归纳法原理：若{$P(1)$，$P(2)$，$P(3)$，$P(4)$，...}是命题序列且满足以下两个性质，则所有命题均为真：

① $P(1)$为真。

② 任何命题均可以从它的*前一个*命题推导得出。

第二数学归纳法原理：若{$P(1)$，$P(2)$，$P(3)$，$P(4)$，...}是满足以下两个性质的命题序列，则对于其他自然数，该命题序列均为真：

① $P(1)$为真。

② 任何命题均可以从它的*前面所有*命题推导得出。

2. 什么是归纳法



归纳法由一系列有限的特殊事例得出一般结论的推理方法

归纳法分为完全归纳法 和 不完全归纳法

完全归纳法	不完全归纳法
考察全体对象，得到一般结论的推理方法	考察部分对象，得到一般结论的推理方法
结论一定可靠	结论不一定可靠

归纳法 ⇨ 递推关系

基本流程：

- 按推导问题方向研究最初最原始的若干问题。
- 按推导问题方向寻求问题间的转换规律即递推关系，使问题逐次转化成较低层级或简单的且能解决问题的或已解决的问题。
- 顺推法和逆推法：顺推法是从已知条件出发逐步推算出要解决问题的结果。逆推法从已知问题的结果出发逐步推算出问题的开始条件。

- 数学归纳法不是归纳法，但它与归纳法有着一定程度的关联。
- 在结论的发现过程中，往往先通过对大量个别事实的观察，通过不完全归纳法归纳形成一般性的结论，最终利用数学归纳法对结论的正确性予以证明。

3.2.2 直接插入排序

- 有一个整数序列$R[0..n-1]$，采用直接插入排序实现R的递增有序排序。
- 直接插入排序的过程是，i从1到$n-1$循环，将$R[i]$有序插入到$R[0..i-1]$中。

解 采用不完全归纳法产生直接插入排序的递推关系。

例如R=（2，5，4，1，3），这里n=5，用[]表示有序区，各趟的排序结果如下：

初始：　（[2]，5，4，1，3）
i=1：　（[2，5]，4，1，3）
i=2：　（[2，4，5]，1，3）
i=3：　（[1，2，4，5]，3）
i=4：　（[1，2，3，4，5]）

设$f(R，i)$用于实现$R[0..i]$（共i+1个元素）的递增排序，它是大问题，则$f(R，i-1)$实现$R[0..i-1]$（共i个元素）的排序，它是小问题。对应的递推关系：

$f(R，i)$≡不做任何事情　　当i=0
$f(R，i)$≡$f(R，i-1)$; 将$R[i]$有序插入到$R[0..i-1]$中；　　其他

$f(R，i)$ ≡ 不做任何事情　　当i=1

$f(R，i)$ ≡ $f(R，i\text{-}1)$; 将$R[i]$有序插入到$R[0..i\text{-}1]$中;　　其他

显然$f(R，n\text{-}1)$用于实现$R[0..n\text{-}1]$的递增排序。

采用不完全归纳法得到的结论是否正确呢？

① 证明归纳基础成立。当n=0时直接返回，由于此时R中只有一个元素，它是递增有序的，所以结论成立。

② 证明归纳递推成立。假设n=k时成立，也就是说$f(R，k\text{-}1)$用于实现$R[0..k\text{-}1]$的递增排序。

当n=k+1时对应$f(R，k)$，先调用$f(R，k\text{-}1)$将$R[0..k\text{-}1]$排序，再将$R[k]$有序插入到$R[0..k\text{-}1]$中，这样$R[0..k]$变成递增有序序列了，所以$f(R，k)$实现$R[0..k]$的递增排序，结论成立。

```
void Insert(vector<int>&R,int i)      //插入操作：将R[i]有序插入到R[0..i-1]中
{   int tmp=R[i];
    int j=i-1;
    do                                //找R[i]的插入位置
    {   R[j+1]=R[j];                  //将关键字大于R[i]的元素后移
        j--;
    } while(j>=0 && R[j]>tmp);        //直到R[j]<=tmp为止
    R[j+1]=tmp;                       //在j+1处插入R[i]
}

void InsertSort1(vector<int> &R)      //直接插入排序
{   int n=R.size();
    for (int i=1;i<n;i++)
    {   if (R[i]<R[i-1])              //反序时
            Insert(R,i);
    }
}
```

插入排序

3.2.3 楼梯问题

- 一个楼梯共有n个台阶，规定每一步只能跨一个或两个台阶。
- 设计一个算法求登上第n级台阶有多少种不同走法。

采用归纳法中的顺推法。设$f(n)$表示登上第n级台阶的不同的走法数。

① 当$n=1$时，只有跨一个台阶的一种走法，即$f(1)=1$。

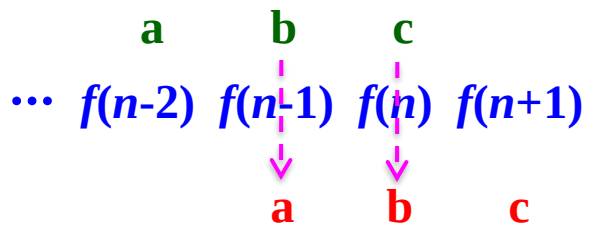
② 当$n=2$时，可以走两步每步跨一个台阶，也可以走一步跨两个台阶，这样共有两种走法，即$f(2)=2$。

③ 当$n=3$时，考虑第一步，第一步跨两个台阶，剩下一个台阶（剩下一个台阶的不同走法数为$f(1)$），对应的不同走法数为$f(1)$；第一步跨一个台阶，剩下2个台阶（剩下2个台阶的不同走法数为$f(2)$），对应的不同走法数为$f(2)$。采用加法原理有$f(3)=f(1)+f(2)$。

④ 当$n=4$时，$f(4)=f(2)+f(3)$。

$$
f(n)=\begin{cases} 1 & n=1 \\ 2 & n=2 \\ f(n-2)+f(n-1) & n>2 \end{cases}
$$

```
int Count(int n)
{    int a=1,b=2,c;
     if(n==1)
          return a;
     else if(n==2)
          return b;
     else
     {    for(int i=3;i<=n;i++)
          {    c=a+b;
               a=b;
               b=c;
          }
          return c;
     }
}
```



爬楼梯问题

3.3 迭代法

3.3.1 迭代法概述

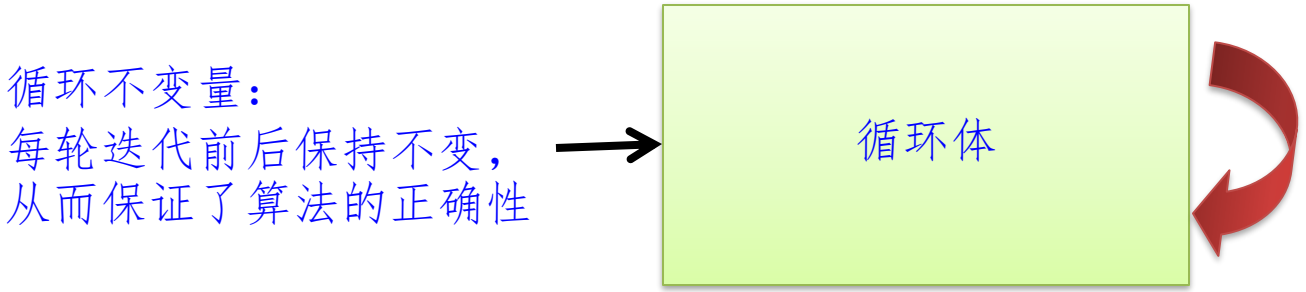
迭代法也称辗转法，是一种不断用变量的旧值推出新值的过程。

通过让计算机对一组指令（或一定步骤）进行重复执行，在每次执行这组指令（或这些步骤）时，都从变量的原值推出它的一个新值。

迭代法算法框架：

```
void Iterative()
{  迭代变量赋初值;
   while (迭代条件成立)
   {  根据递推关系式由旧值计算出新值;
      新值取代旧值，为下一次迭代做准备;
   }
}
```

- 迭代法算法包含循环，对循环的证明引入循环不变量的概念。
- 循环不变量是指在每轮迭代开始前后要操作的数据必须保持的某种特性（比如在直接插入排序中，排序表前面部分必须是有序的）。
- 循环不变量是进行循环的必备条件，因为它保证了循环进行的有效性，有助于理解算法的正确性。



循环不变量必须证明它的三个性质：

- 初始化：在循环的第一轮迭代开始之前，应该是正确的。
- 保持：如果循环的第一次迭代开始之前正确，那么在下一次迭代开始之前它也应该保持正确。
- 终止：当循环结束时，循环不变量给了我们一个有用的性质，它有助于表明算法是正确的。

【例3-5】采用循环不变量方法证明3.2.2节中直接插入排序算法的正确性。

证明：直接插入排序算法中循环不变量为$R[0..i-1]$是递增有序的。

初始化：循环时i从1开始，循环之前$R[0..0]$只有一个元素，显然成立。

保持：需要证明每一轮循环都能使循环不变量保持成立。对于$R[i]$排序的这一趟，之前$R[0..i-1]$是递增有序的：

① 如果$R[i] \geq R[i-1]$即正序，则该趟结束，结束后循环不变量$R[0..i]$显然是递增有序的。

② 如果$R[i] < R[i-1]$即反序，则在$R[0..i-1]$中从后向前找到第一个$R[j] \leq R[i]$，将$R[j+1..i-1]$均后移一个位置，并且将原$R[i]$放在$R[j+1]$位置，这样结束后循环不变量$R[0..i]$显然也是递增有序的。

终止：循环结束时$i=n$，在循环不变量中用i替换n，就有$R[0..n-1]$包含原来的全部元素，现在已经排好序了，即说循环不变量也是成立的。

3.3.2 简单选择排序

- 有一个整数序列$R[0..n-1]$，采用简单选择排序实现R的递增有序排序。
- 简单选择排序的过程是：i从0到$n-2$循环，$R[0..i-1]$是有序区，$R[i..n-1]$是无序区，并且前者的所有元素均小于等于后者的任意元素，在$R[i..n-1]$无序区通过简单比较找到最小元素$R[minj]$，通过交换将其放在$R[i]$位置。

解

- 采用不完全归纳法产生简单选择排序的递推关系。
- 例如$R=$（2，5，4，1，3），这里$n=5$，用[]表示有序区，各趟的排序结果如下：

初始：	（[]2，5，4，1，3）
$i=0$：	（[1]，5，4，2，3）
$i=1$：	（[1，2]，4，5，3）
$i=2$：	（[1，2，3]，5，4）
$i=3$：	（[1，2，3，4]，5）

设*f*(*R*，*i*)用于实现*R*[*i*..*n*-1]（共*n*-*i*个元素）的递增排序，它是大问题，
则*f*(*R*，*i*-1) 实现*R*[*i*-1..*n*-1]（共*n*-*i*-1个元素）的排序，它是小问题。

f(*R*，*i*) ≡ 不做任何事情　　当*i*=*n*-1时

f(*R*，*i*) ≡ 在*R*[*i*..*n*-1]中选择最小元素交换到*R*[*i*]位置;　　否则

　　f(*R*，*i*+1);

算法实现：

```
void Select(vector<int>& R,int i)     //选择操作：R[i..n-1]选最小元素交换到R[i]
{    int minj=i;                      //minj表示R[i..n-1]中最小元素的下标
     for (int j=i+1;j<R.size();j++)   //在R[i..n-1]中找最小元素
          if (R[j]<R[minj])
               minj=j;
     if (minj!=i)                     //若最小元素不是R[i]
          swap(R[minj],R[i]);         //交换
}
void SelectSort1(vector<int>& R)      //迭代法：简单选择排序
{    int n=R.size();
     for (int i=0;i<n-1;i++)          //进行n-1趟排序
          Select(R,i);
}
```

选择排序

3.3.5 求幂集

给定正整数n（$n \geqslant 1$），求{1～n}的幂集的递归模型和迭代算法。

例如，n=3时，{1，2，3}的幂集合为{{}，{1}，{2}，{1，2}，{3}，{1，3}，{2，3}，{1，2，3}}。

解 以n=3为例，求1～3的幂集的过程

1的幂集M_1：{{},{1}}

↓ M_1中每个集合元素添加2得到A_2

A_2：{{2},{1,2}}

↓ $M_2=M_1 \cup A_2$

1～2的幂集M_2：{{},{1},{2},{1,2}}

↓ M_2中每个集合元素添加3得到A_3

A_3：{{3},{1,3},{2,3},{1,2,3}}

↓ $M_3=M_2 \cup A_3$

1～3的幂集M_3：{{},{1},{2},{1,2},{3},{1,3},{2,3},{1,2,3}}

定义运算**appendi(M_{i-1}，*i*)**返回在M_{i-1}中每个集合元素末尾插入整数*i*的结果

$$\mathrm{appendi}(\mathrm{M}_{i-1}, i) = \bigcup_{\mathrm{s}\in \mathrm{M}_{i-1}} \mathrm{push_back}(\mathrm{s}, i)$$

这样求**{1～*n*}**的幂集的递推关系如下：

M_1={{}，{1}}

$M_i=M_{i-1}\cup A_i$　　当*i*>1时

```
vector<vector<int>> appendi(vector<vector<int>> Mi_1,int i)
//向Mi_1中每个集合元素末尾添加i
{    vector<vector<int>> Ai=Mi_1;
     for(int j=0;j<Ai.size();j++)
          Ai[j].push_back(i);
     return Ai;
}

vector<vector<int>> subsets1(int n)          //迭代法：求{1～n}的幂集
{    vector<vector<int>> Mi;                 //存放{1～n}的幂集
     vector<vector<int>> Mi_1={{},{1}};      //初始时存放M1
     if(n==1) return Mi_1;                   //处理特殊情况
     for(int i=2;i<=n;i++)                   //迭代循环
     {    vector<vector<int>> Ai=appendi(Mi_1,i);
          Mi=Mi_1;
          for(int j=0;j<Ai.size();j++)       //将Ai所有集合元素添加到Mi中
               Mi.push_back(Ai[j]);
          Mi_1=Mi;                           //新值取代旧值
     }
     return Mi;
}
```

迭代法求幂集

3.4 递归法

3.4.1 递归法概述

1. 什么是递归

- 递归算法是指在算法定义中又调用自身的算法。
- 递归算法通常把一个大的复杂问题层层转化为一个或多个与原问题相似的规模较小的问题来求解，具有思路清晰和代码少的优点。一般来说，能够用递归解决的问题应该满足以下3个条件：

① 原问题可以转化为一个或多个子问题来求解，而这些子问题的求解方法与原问题完全相同，只是在数量规模上不同。

② 递归调用的次数必须是有限的。

③ 必须有结束递归的条件来终止递归。

2. 递归模型

简化的递归模型：$f(s_n) \Rightarrow f(s_{n-1})$

$f(s_1)=m$

$f(s_n)=g(f(s_{n-1}),\ c)$

3. 提取求解问题的递归模型

① 对大问题$f(s)$进行分析，假设出合理的“小问题”$f(s')$。

② 假设小问题$f(s')$是可解的，在此基础上确定大问题$f(s)$的解，即给出$f(s)$与$f(s')$之间的递推关系，也就是提取递归体（与数学归纳法中假设$i=n-1$时等式成立，再求证$i=n$时等式成立的过程相似）。

③ 确定一个特定情况（如$f(1)$或$f(0)$）的解，由此作为递归出口（与数学归纳法中求证$i=1$或$i=0$时等式成立相似）。

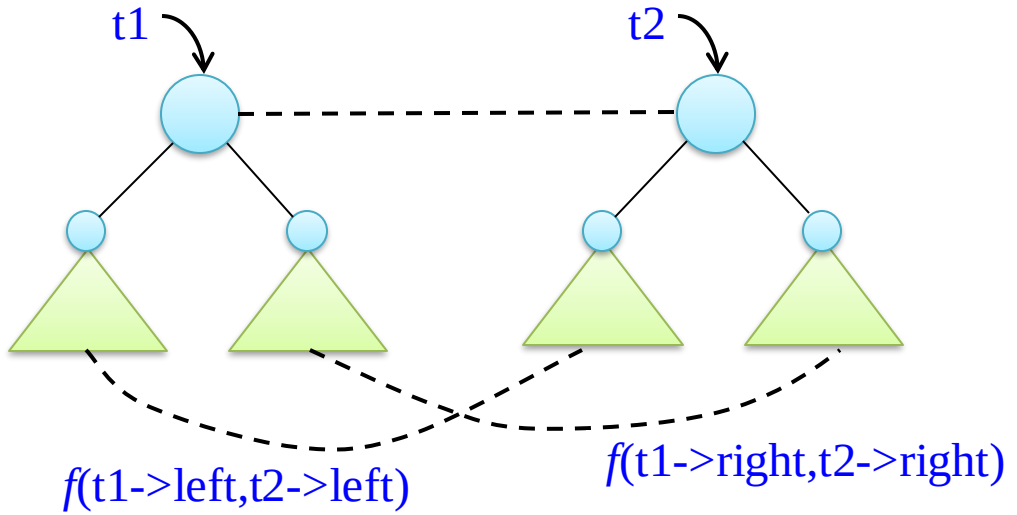
4. 递归算法框架

```
void recursion1(n)              //先递后合的递归框架
{   if (满足出口条件)
       直接解决;
    else
    {   recursion1(m);          //递去，递到最深处
        merge();                //归来时执行合并操作
    }
}

void recursion2(n)              //先合后递的递归框架
{   if (满足出口条件)
      直接解决;
    else
    {   merg();                 //合并：递去
        recursion2(m);          //递到最深处后，再不断地归来
    }
}
```

【例3-6】假设二叉树采用二叉链存储，设计一个算法判断两棵二叉树t1和t2是否相同，所谓相同是指它们的形态相同并且对应的结点值相同。

解 设*f*(t1，t2)表示二叉树t1和t2是否相同，它们的左右子树的判断是两个小问题。



f(t1，t2)=true
f(t1，t2)=false
f(t1，t2)=false
f(t1，t2)=*f*(t1->left,t2->left) && *f*(t1->right,t2->right)

当t1和t2均为空
当t1、t2中一空一非空
当均不空但结点值不同
其他

```
bool same(TreeNode* t1,TreeNode* t2)        //递归算法：判断t1和t2是否相同
{     if(t1==NULL && t2==NULL)
          return true;
      else if(t1==NULL || t2==NULL)
          return false;
      if(t1->val!=t2->val)
          return false;
      bool left=same(t1->left,t2->left);          //递归调用1
      bool right=same(t1->right,t2->right);       //递归调用2
      return left && right;
}
```

递归算法判断两颗子树是否相同

3.4.2 冒泡排序

- 有一个整数序列$R[0..n-1]$，采用冒泡排序实现R的递增有序排序。
- 冒泡排序的过程是，i从0到$n-2$循环，$R[0..i-1]$是有序区，$R[i..n-1]$是无序区，并且前者的所有元素均小于等于后者的任意元素，在$R[i..n-1]$无序区通过冒泡方式将最大元素放在$R[i]$位置。

解

- 采用不完全归纳法产生冒泡排序的递推关系。
- 例如$R=$（2，5，4，1，3），这里$n=5$，用[]表示有序区，各趟的排序结果如下：

初始：	（[]2，5，4，1，3）
$i=0$：	（[1]，2，5，4，3）
$i=1$：	（[1，2]，3，5，4）
$i=2$：	（[1，2，3]，4，5）
$i=3$：	（[1，2，3，4]，5）

先递后合

设$f(R，i)$用于实现$R[0..i]$（共$i+1$个元素）的递增排序，它是大问题，则$f(R，i-1)$ 实现$R[0..i-1]$（共i个元素）的排序，它是小问题。当$i=-1$时，$R[0..i]$为空，看成是有序的。

$f(R，i)$ ≡ 不做任何事情　　当$i=-1$

$f(R，i)$ ≡ $f(R，i-1)$;　　否则

　　在$R[i..n-1]$中冒泡最小元素到$R[i]$位置;

```
void Bubble1(vector<int>& R,int i)          //冒泡操作：R[i..n-1]冒最小元素R[i]
{    int n=R.size();
     for (int j=n-1;j>i;j--)                //无序区元素比较,找出最小元素
          if (R[j-1]>R[j])                  //当相邻元素反序时
               swap(R[j],R[j-1]);           //R[j]与R[j-1]进行交换
}

void BubbleSort21(vector<int>& R,int i)     //被BubbleSort2调用
{    if (i==-1) return;                     //满足递归出口条件
     BubbleSort21(R,i-1);                   //递归调用
     Bubble1(R,i);                          //合并操作
}

void BubbleSort2(vector<int>& R)            //递归算法：冒泡排序
{    int n=R.size();
     BubbleSort21(R,n-2);
}
```

先合后递

设$f(R，i)$用于实现$R[i..n-1]$（共$n-i$个元素）的递增排序，它是大问题，则$f(R，i-1)$ 实现$R[i-1..n-1]$（共$n-i-1$个元素）的排序，它是小问题。当$i=n-1$时，$R[n-1..n-1]$仅包含最后一个元素，它一定是最大元素，排序结束。

$f(R，i)$ ≡ 不做任何事情　　当$i=n-1$

$f(R，i)$ ≡ 在$R[i..n-1]$中冒泡最小元素到$R[i]$位置;　　否则

　　$f(R，i+1)$;

```
void BubbleSort31(vector<int>& R,int i)      //被BubbleSort3调用
{    int n=R.size();
     if (i==n-1) return;                     //满足递归出口条件
     Bubble1(R,i);                           //合并操作
     BubbleSort31(R,i+1);                    //递归调用
}

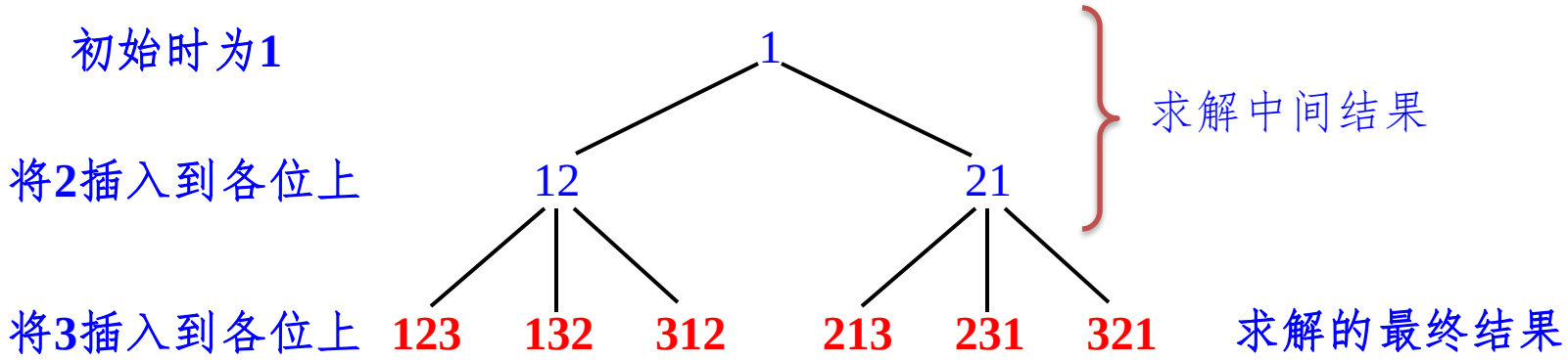
void BubbleSort3(vector<int>& R)             //递归算法：冒泡排序
{
     BubbleSort31(R,0);
}
```

这里很脑瘫，为什么要把冒泡算法强行用递归的形式写呢，这不是闲的有病吗

3.4.3 求全排列

- 给定正整数n（$n \geqslant 1$），给出求1～n的全排序的递归模型和递归算法。
- 例如，n=3时，全排列是{{1，2，3}，{1，3，2}，{3，1，2}，{2，1，3}，{2，3，1}，{3，2，1}}。

解　以n=3为例，求1～3的全排列的步骤。



插入递归求全排列

设P_i表示1～i（$i \geq 1$，共i个元素）的全排列（是一个两层集合，其中每个集合元素表示1～i的某个排列），为大问题，则P_{i-1}为1～i-1（共i-1个元素）的全排列，为小问题。显然有P_1={{1}}。

$$\text{Createpi}(s,i) = \bigcup_{0 \leq j \leq s.\text{size}()} \text{Insert}(s,i,j)$$

$$P_i = \bigcup_{s \in P_{i-1}} \text{Createpi}(s,i)$$

P_1={{1}}

$P_i = \bigcup_{s \in P_{i-1}} \text{Createpi}(s,i)$ 当i>1时

```
vector<int> Insert(vector<int> s,int i,int j)          //在s的位置j插入i
{   vector<int>::iterator it=s.begin()+j;              //求出插入位置
    s.insert(it,i);                                    //插入整数i
    return s;
}
```

$$\text{Createpi}(s,i)=\bigcup_{0\le j\le s.size()}\text{Insert}(s,i,j)$$

```
vector<vector<int>> CreatePi(vector<int> s,int i)   //在s集合中i-1到0位置插入i
{   vector<vector<int>> tmp;
    for (int j=s.size();j>=0;j--)                   //在s(含i-1个整数)每个位置插入i
    {   vector<int> s1=Insert(s,i,j);
        tmp.push_back(s1);                          //添加到Pi中
    }
    return tmp;
}
```

```
vector<vector<int>> perm21(int n,int i)                    //被Perm2调用
{    if(i==1)                                              //递归出口
         return {{1}};
     else
     {    vector<vector<int>> Pi;
          vector<vector<int>> Pi_1=perm21(n,i-1);          //求出Pi_1
          for (auto it=Pi_1.begin();it!=Pi_1.end();it++)   //Pi每个元素的各位置插入i
          {    vector<vector<int>> tmp=CreatePi(*it,i);
               for(int k=0;k<tmp.size();k++)
                    Pi.push_back(tmp[k]);                  //将tmp全部元素添加到Pi中
          }
          return Pi;
     }
}

vector<vector<int>> Perm2(int n)                           //递归法求1～n的全排列
{
     return perm21(n,n);
}
```

3.5 递推式计算

3.5.1 直接展开法

- 求解递推式最自然的方法是将其反复展开。
- 即直接从递归式出发，一层一层地往前递推，直到最前面的初始条件为止，就得到了问题的解。

【例3-8】求解梵塔问题的递归算法如下，分析移动n盘片的时间复杂度。

解

```
void Hanoi(int n,char x,char y,char z)
{   if (n==1)
        printf("将盘片%d从%c搬到%c\n",n,x,z);
    else
    {   Hanoi(n-1,x,z,y);
        printf("将盘片%d从%c搬到%c\n",n,x,z);
        Hanoi(n-1,y,x,z);
    }
}
```

效率分析：

Hanoi(n，x，y，z)的执行时间为$T(n)$

$$T(n)=\begin{cases}1 & \text{当 } n=1\\ 2T(n-1)+1 & \text{当 } n>1\end{cases}$$

$$T(n)=2[2T(n-2)+1]+1=2^2T(n-2)+1+2^1$$
$$=2^3T(n-3)+1+2^1+2^2$$
$$=\cdots$$
$$=2^{n-1}T(1)+1+2^1+2^2+\cdots+2^{n-2}$$
$$=2^n-1=O(2^n)$$

三层汉诺塔计算时间复杂度

3.5.2 递归树方法

- 用递归树求解递推式的基本过程是，展开递推式，构造对应的递归树，然后把每一层的时间进行求和，从而得到算法执行时间的估计。
- 再用时间复杂度形式表示。

第三部分

本章结束！

主讲人：孙岐峰

联系电话：13864717907

Q Q号码：274342509

邮　　箱：sunqf@upc.edu.cn